

EJERCICIO EMPRESA CON AGREGACION

CLASE EMPRESA

```
package com.ejemplo.ejercicioempresa;

//Importa la clase ArrayList de Java. La necesitas porque vas a guardar varios objetos
//Departamento dentro de una lista.
import java.util.ArrayList;

public class Empresa {

    private String nombre;
    private String nit;
    private String ciudad;

    // AGREGACIÓN
    //declaras una lista que puede guardar objetos de tipo Departamento.
    private ArrayList<Departamento> departamentos;

    public Empresa(String nombre, String nit, String ciudad) {

        if (nombre == null || nombre.trim().isEmpty()) {
            //throw significa lanzar una excepción.
            //IllegalArgumentException indica que se recibió un argumento incorrecto.
            throw new IllegalArgumentException(
                "El nombre de la empresa no puede estar vacío");
        }
    }
}
```

```

        if (nit == null || nit.trim().isEmpty()) {
            throw new IllegalArgumentException(
                "El nit de la empresa no puede estar vacío");
        }

        if (ciudad == null || ciudad.trim().isEmpty()) {
            throw new IllegalArgumentException(
                "La ciudad de la empresa no puede estar vacía");
        }

        this.nombre = nombre;
        this.nit = nit;
        this.ciudad = ciudad;

        // MUY IMPORTANTE
        //estás creando la lista de departamentos.
        departamentos = new ArrayList<>();
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getNit() {
        return nit;
    }

```

```
public void setNit(String nit) {  
    this.nit = nit;  
}
```

```
public String getCiudad() {  
    return ciudad;  
}
```

```
public void setCiudad(String ciudad) {  
    this.ciudad = ciudad;  
}
```

// MÉTODO DE AGREGACIÓN

//Este método recibe un objeto de tipo: departamento

```
public void agregarDepartamento(Departamento departamento) {  
  
    if (departamento == null) {  
        throw new IllegalArgumentException(  
            "El departamento no puede ser nulo");  
    }  
    //Agregar un elemento al ArrayList.  
    departamentos.add(departamento);  
}
```

```
public void mostrarDepartamentos() {
```

```
    System.out.println("Departamentos de la empresa " + nombre + ":");
```

```
    //Este tipo de for se llama for-each o for mejorado.
```

//Sirve para recorrer uno por uno todos los elementos que están guardados en una
//colección,

//en esta caso el ArrayList de departamentos.

```
for (Departamento departamento : departamentos) {  
    System.out.println("- " + departamento.getNombre());  
}  
}
```

```
public void mostrar() {
```

```
    System.out.println(  
        "Nombre de la empresa: " + nombre);
```

```
    System.out.println(  
        "Nit de la empresa: " + nit);
```

```
    System.out.println(  
        "Ciudad: " + ciudad);
```

```
    }  
}
```

CODIGO CLASE DEPARTAMENTO

```
package com.ejemplo.ejercicioempresa;

import java.util.ArrayList;

public class Departamento {

    private String nombre;
    private String codigo;
    private Empresa empresa;

    // AGREGACIÓN: un departamento tiene varios empleados
    private ArrayList<Empleado> empleados;

    public Departamento(String nombre, String codigo, Empresa empresa) {

        if (nombre == null || nombre.trim().isEmpty()) {
            throw new IllegalArgumentException(
                "El nombre no puede estar vacío");
        }

        if (codigo == null || codigo.trim().isEmpty()) {
            throw new IllegalArgumentException(
                "El codigo no puede estar vacío");
        }

        if (empresa == null) {
            throw new IllegalArgumentException(
                "La empresa no puede estar vacía");
        }
    }
}
```

```
}

this.nombre = nombre;
this.codigo = codigo;
this.empresa = empresa;

empleados = new ArrayList<>();
}

public String getNombre() {
    return nombre;
}

public String getCodigo() {
    return codigo;
}

public Empresa getEmpresa() {
    return empresa;
}

public void agregarEmpleado(Empleado empleado) {

    if (empleado == null) {
        throw new IllegalArgumentException(
            "El empleado no puede ser nulo");
    }

    empleados.add(empleado);
}
```

```
public void mostrarEmpleados() {
```

```
    System.out.println("Empleados del departamento " + nombre + ":");
```

```
    for (Empleado empleado : empleados) {
```

```
        System.out.println("- " + empleado.getNombre());
```

```
    }
```

```
}
```

```
public void mostrardepartamento() {
```

```
    System.out.println(
```

```
        "Nombre del Departamento: " + nombre);
```

```
    System.out.println(
```

```
        "Codigo del Departamento: " + codigo);
```

```
    System.out.println(
```

```
        "Nombre de la empresa: " + empresa.getNombre());
```

```
    }
```

```
}
```

CODIGO CLASE EMPLEADOS

```
package com.ejemplo.ejercicioempresa;
```

```
public class Empleado {
```

```
    // ATRIBUTOS ENCAPSULADOS
```

```
    private String nombre;
```

```
    private String documento;
```

```
    private float salario;
```

```
    private Departamento departamento;
```

```
    // CONSTRUCTOR
```

```
    public Empleado(String nombre, String documento,float salario, Departamento  
departamento) {
```

```
        if (nombre == null || nombre.trim().isEmpty()) {  
            throw new IllegalArgumentException(  
                "El nombre no puede estar vacío");  
        }
```

```
        if (documento == null || documento.trim().isEmpty()) {  
            throw new IllegalArgumentException(  
                "El documento no puede estar vacío");  
        }
```

```
        if (salario <= 0) {  
            throw new IllegalArgumentException(  
                "El salario debe ser mayor que cero");  
        }
```

```
if (departamento == null) {  
    throw new IllegalArgumentException(  
        "El departamento no puede estar vacío");  
}
```

```
    //Asignación de valores  
    this.nombre = nombre;  
    this.documento = documento;  
    this.salario = salario;  
    this.departamento = departamento;  
  
}
```

```
// GET Y SET DE NOMBRE
```

```
public String getNombre() {  
    return nombre;  
}
```

```
public void setNombre(String nombre) {  
    this.nombre = nombre;  
}
```

```
// GET Y SET DE DOCUMENTO
```

```
public String getDocumento() {  
    return documento;  
}
```

```
public void setDocumento(String documento) {
```

```
    this.documento = documento;  
}
```

```
// GET Y SET DE SALARIO
```

```
public float getSalario() {  
    return salario;  
}
```

```
public void setSalario(float salario) {
```

```
    this.salario = salario;  
}
```

```
// GET Y SET DE DEPARTAMENTO
```

```
public Departamento getDepartamento() {  
    return departamento;  
}
```

```
public void setDepartamento(Departamento departamento) {
```

```
    this.departamento = departamento;  
}
```

```
// MÉTODO MOSTRAR
```

```
public void mostrareempleados() {
```

```
    System.out.println("Nombre del empleado: " + nombre);
```

```
    System.out.println("Cedula del empleado: " + documento);
```

```
    System.out.println("Salario: " + salario);
```

```
    System.out.println("Departamento donde trabaja: " + departamento.getNombre()  
    );
```

```
    System.out.println("Empresa donde trabaja: " +  
    departamento.getEmpresa().getNombre());
```

```
    }  
}
```

CODIGO CLASE MAIN (PRINCIPAL)

```
package com.ejemplo.ejercicioempresa;
```

```
public class Ejercicioempresa {
```

```
    public static void main(String[] args) {
```

```
        // Declaración de variables
```

```
        Empresa empresa1 = null;
```

```
        Departamento departamento1 = null;
```

```
        Departamento departamento2 = null;
```

```
        Empleado empleado1 = null;
```

```
        Empleado empleado2 = null;
```

```
        Empleado empleado3 = null;
```

```
        // =====
```

```
        // CREAR EMPRESA
```

```
        // =====
```

```
        try {
```

```
            empresa1 = new Empresa("ALPINA", "800.154.235-8", "Bogota");
```

```
            System.out.println("-----");
```

```
            empresa1.mostrar();
```

```
        } catch (IllegalArgumentException e) {
```

```

        System.out.println("ERROR EMPRESA: " + e.getMessage());
    }

    // =====
    // CREAR DEPARTAMENTO 1
    // =====

    if (empresa1 != null) {

        try {

            departamento1 = new Departamento("Sistrmas","A001",empresa1);

        } catch (IllegalArgumentException e) {

            System.out.println("ERROR DEPARTAMENTO 1: " + e.getMessage());
        }

    } else {

        System.out.println("No se puede crear el departamento " + "porque la empresa no
existe.");
    }

    // =====
    // CREAR DEPARTAMENTO 2
    // =====

```

```
if (empresa1 != null) {

    try {

        departamento2 = new Departamento("Contabilidad","B001",empresa1);

    } catch (IllegalArgumentException e) {

        System.out.println("ERROR DEPARTAMENTO 2: " + e.getMessage());
    }

}
```

```
// =====
// AGREGAR DEPARTAMENTOS A LA EMPRESA
// =====
```

```
if (empresa1 != null) {

    try {

        if (departamento1 != null) {
            empresa1.agregarDepartamento(
                departamento1);
        }

        if (departamento2 != null) {
            empresa1.agregarDepartamento(
```

```

        departamento2);
    }

    System.out.println("-----");
    empresa1.mostrarDepartamentos();

} catch (IllegalArgumentException e) {

    System.out.println("ERROR AL AGREGAR DEPARTAMENTO: " +
e.getMessage());
}
}

// =====
// CREAR EMPLEADO 1
// =====

if (departamento1 != null) {

    try {

        empleado1 = new Empleado("Juan","1098765432",2500000,departamento1);

    } catch (IllegalArgumentException e) {

        System.out.println("ERROR EMPLEADO 1: " + e.getMessage());
    }
}
}

```

```
// =====  
// CREAR EMPLEADO 2  
// =====  
  
if (departamento1 != null) {  
  
    try {  
  
        empleado2 = new Empleado("Pablo","77777",1800000,departamento1);  
  
    } catch (IllegalArgumentException e) {  
  
        System.out.println("ERROR EMPLEADO 2: " + e.getMessage());  
    }  
}  
  
// =====  
// CREAR EMPLEADO 3  
// =====  
  
if (departamento2 != null) {  
  
    try {  
  
        empleado3 = new Empleado("Sandra","66666",2200000,departamento2);  
  
    } catch (IllegalArgumentException e) {
```

```

        System.out.println("ERROR EMPLEADO 3: " + e.getMessage());
    }
}

// =====
// AGREGAR EMPLEADOS
// =====

try {

    if (departamento1 != null
        && empleado1 != null) {

        departamento1.agregarEmpleado(empleado1);
    }

    if (departamento1 != null
        && empleado2 != null) {

        departamento1.agregarEmpleado(empleado2);
    }

    if (departamento2 != null
        && empleado3 != null) {departamento2.agregarEmpleado(empleado3);
    }

} catch (IllegalArgumentException e) {

    System.out.println("ERROR AL AGREGAR EMPLEADO: " + e.getMessage());
}

```

```
}
```

```
// =====
```

```
// MOSTRAR EMPLEADOS
```

```
// =====
```

```
if (departamento1 != null) {
```

```
    System.out.println("-----");
```

```
    departamento1.mostrarEmpleados();
```

```
}
```

```
if (departamento2 != null) {
```

```
    System.out.println("-----");
```

```
    departamento2.mostrarEmpleados();
```

```
}
```

```
}
```

```
}
```